Decision Validation: Trade Plan to Confirmable Scenarios

Current implementation • JSON + RAG + dual-channel AI

01 INPUT

Trade plan

Asset / action / amount
Rationale / horizon

JSON contract:

`input.v1`

02 ORCHESTRATE

Parse rationale

Structured AI JSON
Facts / claims / inference

Local-rule fallback

03 VERIFY

Evidence + RAG

Prior disclosure check
Financial cross-checks

64D hybrid retrieval

04 FORECAST

Dual forecasts

A. Public evidence
B. 260-day OHLCV

Isolated source lists

05 PRESENT

Review result

Downside / base / upside
1 / 3 / 6 / 12 months
Curves + tables

User confirmation

Quality gate - output reaches the UI only after JSON validation probabilities = 100 • all horizons present • distinct paths • return bounds $[-100, 200]$ • strict evidence/technical source isolation

`ai_rag`	`partial_ai_rag`	`rules`
Both AI channels pass	Independent channel fallback	Bounded scenarios + warning

Safety boundary No buy/sell command or return guarantee • `allow_signal=false` • final state remains `awaiting_confirmation`

Where AI Participates and What It Contributes

Three bounded AI tasks inside a deterministic validation pipeline

AI-1 INTERPRET

Rationale parsing

Structure free-text rationale
Separate facts / claims / inferences

Expose assumptions and invalidation conditions

Value: auditable inputs

CONTROL GROUND

Assemble context

Verified public evidence
Financial cross-checks
64D hybrid method RAG

260-day OHLCV summary

Purpose: constrain model context

AI-2 + AI-3 FORECAST

Independent channels

Evidence AI: links verified sources to conditional downside, base, and upside drivers.

Technical AI: links trend, momentum, and volatility to independent paths.

Value: two explainable views

CONTROL VALIDATE

Gate + present

Enforce schema, horizons, probability total, and bounds
Apply channel fallback

Render curves and tables

State: awaiting confirmation

What AI adds

Semantic interpretation • conditional reasoning • evidence synthesis • uncertainty-aware explanations

What AI cannot control

Source allowlists • validation arithmetic • fallback policy • final confirmation • buy/sell signals

The model proposes conditional scenarios; deterministic code decides whether those scenarios are admissible.

Conversation-Driven Build: Scope to Tested Product

User feedback → code evidence → implementation → verification

01 SCOPE + BRANCH

User: Stay on `Decision-function`.

Build: Inspect live code; keep only Pre-trade review; do not merge the old UI.

02 CORE CAPABILITY

User: Forecast post-trade asset value.

Build: Add versioned JSON contracts and 1/3/6/12-month conditional paths.

03 AI + RAG

User: Use AI with forecasting references.

Build: Add forecast JSON, method RAG, 64D retrieval, and source refs.

04 SOURCE BOUNDARIES

User: Use verified public evidence plus K-line analysis.

Build: Run isolated AI calls with allowlists, fallbacks, and a quality gate.

05 UI CONVERGENCE

User: Reduce inputs; keep Pre-trade only; add curves.

Build: Create one-click review, curves + tables, and direct navigation.

06 TEST + ITERATE

User: Test I/O, set a goal, keep iterating.

Build: Exercise real AI, invalid inputs, builds, desktop, and 390px UI.

Iteration loop feedback → code evidence → scoped change → JSON/API checks → responsive UI review → regression suite

3 / 3 real AI scenarios | **4 / 4 invalid inputs returned 400** | **148 / 148 automated tests** | **Local only, no push**